

Model Solution to Mid Semester Examination, Nov-2025

Practical Programming with C (CSE 3544)

Programme: B.Tech.(CSE)

Semester: 5th

Full marks: 30

Time: 2 hours

Subject Learning Outcome	*Taxonomy Level	Question Number	Marks
Write, compile and execute C programs to solve computational problems.	L2,L3,L4,L5	1a,1b,1c	2,2,2
become familiar with data structures, sorting, searching and string processing.	L2,L3,L4,L5	2a,2b,2c, 3a,3b,3c	2,2,2, 2,2,2
work with preprocessor directives and advance data structures using functions pointers.	L2,L3,L4,L5	4a,4b,4c	2,2,2
understand processes, threads and concurrent execution of processes/threads.	L1,L2,L4,L5	5a,5b,5c	2,2,2
learn file IO, buffered IO, inter-process communication and advanced IO.			
demonstrate the usages of C in graphs, graphics and apply security in coding			

*Bloom's taxonomy levels: Knowledge (L1), Comprehension (L2), Application (L3), Analysis (L4), Evaluation (L5), Creation (L6)

Answer all questions. All questions carry equal marks. All bits of each question carry equal marks.

The figures in the right side indicate marks.

[Note: We are aware of the fact that the solutions of writing program/code snippet related questions are many. So, consider the given solution for a particular program/code snippet as one of the probable solution only. Please don't be strict to the given solutions.]

1. (a) Determine and explain the output of the following code.

[2]

```
int main() { char ch='A';
printf("%-6d", sizeof(ch));
printf("%03d", sizeof('A'));
return(0);
}
```

ANS :

1 004

Explanation:

Here, `sizeof(ch)` returns size of character variable which is 1 byte and displayed with `%-6d` that indicates total 6-character spaces are used to display 1 with right padding of blank spaces. Similarly, `sizeof('A')` returns size of 65 (ASCII value of 'A') which is equivalent to size of int i.e. 4 bytes, and when displayed with `%03d`, it takes total 3 digit spaces with left padding of zeros.

- (b) **Determine the output of the code snippet that uses the uppercase conversion characters in the printf function.** [2]

```
int main() {
    int x=0x80ec;
    float y=0.3e-12;
    printf(":%#4x: %#10.2e:\n", x,y);
    printf(":%#4x: %#10.2E:\n", x,y);
    return 0; }
```

ANS :

```
:0x80ec:    3.00e-13:
:0x80ec:    3.00E-13:
```

- (c) **Write a C program to find the number of digits in a given integer.** [2]

ANS :

```
#include <stdio.h>
int main() {
    int n;
    int c = 0;
    printf("Enter an integer: ");
    scanf("%d", &n);
    do {
        n /= 10;
        c++;
    } while (n != 0);
    printf("Number of digits: %d\n", c);
    return 0;
}
```

2. (a) **Design a function with prototype; `void sumarr(int a[], int b[], int r[], int size);` that takes 4 parameters, two int arrays as input arguments, 1 array as output arguments and their effective size respectively to produce a resultant array containing the sums of corresponding array elements a and b. For example, for the three-element input arrays 5 -1 7 and 2 4 -2, the result would be an array containing 7 3 5.** [2]

ANS :

```
void sumarr (int a[],int b[],int r[],int size) {
    for (int i = 0; i < size; i++) {
        r[i] = a[i] + b[i];
    }
}
```

- (b) What is the output of the above program if the base address of a is 100? Explain your answer. [2]

```
#include<stdio.h>
int main() {
    int arr[4][5],i,j;
    for(i=0;i<4;i++){
        for(j=0;j<5;j++){
            arr[i][j]=10*i+j;
        }
    }
    printf("%u\n", arr[2][4]);
    printf("%u\n", arr+1);
    printf("%u\n", arr[2]);
    printf("%u\n", *(arr + 2) + 7);
    return 0;}

```

ANS :

```
24
120
140
168
```

Explanation:

Values of $arr[][]$

Base address=100

0	1	2	3	4
10	11	12	13	14
20	21	22	23	24
30	31	32	33	34

Expression	Meaning	Output
<code>arr[2][4]</code>	Value at 3rd row, 5th column	24
<code>arr + 1</code>	Address of 2nd row = <code>&arr[1][0]</code>	120
<code>arr[2]</code>	Address of 3rd row = <code>&arr[2][0]</code>	140
<code>*(arr + 2) + 7</code>	7 integer blocks past <code>arr[2][0]</code> = $140 + 7 \times 4$	168

- (c) Determine and explain the output of the following code. [2]

```

void f(int a[2][ ])
{
    a[0][1] = 3;
    int i = 0, j = 0;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            printf("%d", a[i][j]);
}
int main() {
    int a[2][3] = {0};
    f(a);
    return 0; }

```

ANS :

Compile Time Error

Explanation:

In the function definition of f(), declaration of the parameter 'a' as multidimensional array requires bounds for all dimensions except the first.

3. (a) **Given a character array**

```
char str[] = {'S', 'O', 'A', 'D', 'U', '\0'};
```

Write a program to reverse the sequence of alphabets only.

[2]

ANS :

```

#include <stdio.h>
#include <string.h>
int main()
{
    char st[] = {'S', 'O', 'A', 'D', 'U', '\0'};
    int left = 0, right = strlen(st) - 1;
    while (left < right) {
        char temp = st[left];
        st[left] = st[right];
        st[right] = temp;
        left++;
        right--;
    }
    printf("Reversed string: %s\n", st);
    return 0;
}

```

(b) **Predict the outcome of the code snippet considering the ASCII values for characters a, e, i, o, r, s, t, z are 97, 101, 105, 111, 114, 115, 116, 122 respectively.** [2]

```
int main(){char c[] = "itersoa";
    char *p=c;
    printf("%s", p+p[1]-p[3]);
}
```

ANS :

ersoa

Explanation:

Array Index	0	1	2	3	4	5	6
Character value	<i>i</i>	<i>t</i>	<i>e</i>	<i>r</i>	<i>s</i>	<i>o</i>	<i>a</i>
ASCII	105	116	101	114	115	111	97

$$p + p[1] - p[3] = p + 116 - 114 = p + 2$$

Hence, the statement:

```
printf("%s", p + 2);
```

Prints the string starting from `p[2]` until the null terminator `'\0'`.(c) **Write a function to return the maximum element of an array.** [2]

ANS :

```
int findMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}
```

4. (a) **Write the output of the following program with justification.** [2]

```
int main(){int *ptr;
    ptr=(int *)realloc(NULL, sizeof(int));
    *ptr=100;
    printf("%d\n", *ptr);
    return 0;}
```

ANS :

100

Explanation:

Calling `realloc()` with a `NULL` pointer behaves like `malloc()`. So, a block of memory to store one integer is allocated dynamically and is pointed to by `ptr`. The statement `*ptr = 100;` stores the integer value 100 into the allocated memory block.

(b) Determine and explain the output of the following code.

[2]

```
int main() {
    struct student{
        int h,w,m; };
    struct student s1={20,40,50};
    struct student *ptr=&s1;
    printf("%d\n",*((int *)ptr+2));
    return 0;}
```

ANS :

50

The variable `s1` of type `struct student` has been assigned the member values `s1.h = 20`, `s1.w = 40`, and `s1.m = 50`. The pointer `ptr` is assigned the address of `s1`.

So, `(int*)ptr` points to the first member `s1.h`, and `(int*)ptr + 2` points to the third member `s1.m`. Finally, `*((int*)ptr + 2)` dereferences the value of `s1.m`, i.e., 50. Thus, the output is 50.

(c) Write a program to declare a Structure inside another Structure. Show how the variables of the inner structure are accessed. [2]

ANS :

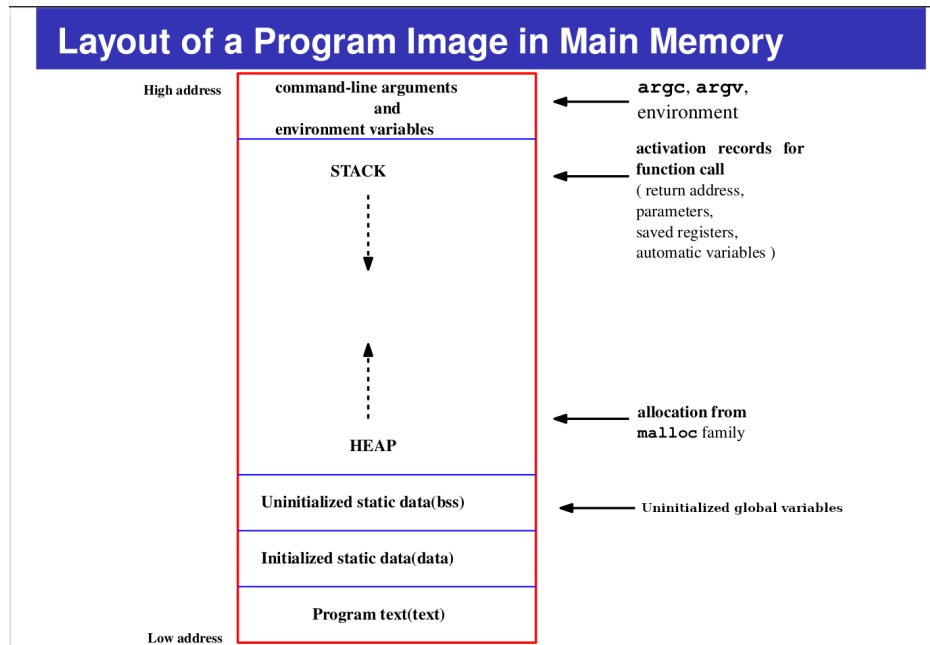
```
//Declaration of a structure inside another structure:
struct date
{
    int month;
    int day;
    int year;
};

struct account{
    int acct_no;
    char acct_type;
    char name[80];
    float balance ;
    struct date lastpayment;
};

//Accessing variables of the inner structure:
int main(){
    struct account acc;
    acc.lastpayment.month=11;
    acc.lastpayment.day=12;
    acc.lastpayment.year=2025;
    return 0;}
```

}

5. (a) **Draw the layout of a program image in main memory and mark all sections. Also, identify which section stores uninitialized global variables and justify your answer. [2]**



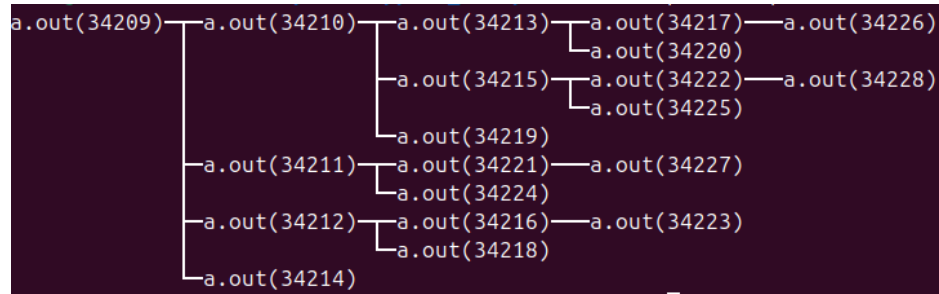
ANS : Uninitialized global and static variables are stored in the BSS (Block Started by Symbol) segment of a program's memory layout.

- (b) **Construct the process tree diagram with the output of the following code snippet, assuming fork never fails. [2]**

```
int main(){
    fork();
    fork() && fork() || fork();
    fork();
    printf("PPWC\n");
    return 0;
}
```

ANS : PPWC is printed 20 times.

Process Tree Diagram:



- (c) Determine the number of process(s) that will be created when the following program becomes a process, and also write the output. [2]

```

void show() {
    if (fork() == 0)
        printf("1\n");
    if (fork() == 0)
        printf("2\n");
    if (fork() == 0)
        printf("3\n");
}

int main(void) {
    show();
    return 0;
}

```

ANS :

```

1    → 1 time
2    → 2 times
3    → 4 times

```

Explanation:

The function `show()` executes 3 separate `if (fork() == 0)` statements.

- The first `fork()` creates 1 new child. Thus, after the first fork:

$$1 \text{ parent } (P) + 1 \text{ child } (A) = 2 \text{ processes}$$

- Both processes (P and A) execute the second `fork()`:

$$2 \text{ processes} \rightarrow 4 \text{ processes total}$$

The resulting processes are:

$$\{P, A, B1, B2\}$$

Children created here: 2. Only the new children print "2\n".

- All 4 processes then execute the third `fork()`:

4 processes $\rightarrow$ 8 processes total

Resulting processes:

$\{P, A, B1, B2, C1, C2, C3, C4\}$

Children created here: 4. These 4 new children print "3\n".

Thus, the total number of processes created is:

1 (original) + 7 (new) = 8.

Process Tree Diagram:

